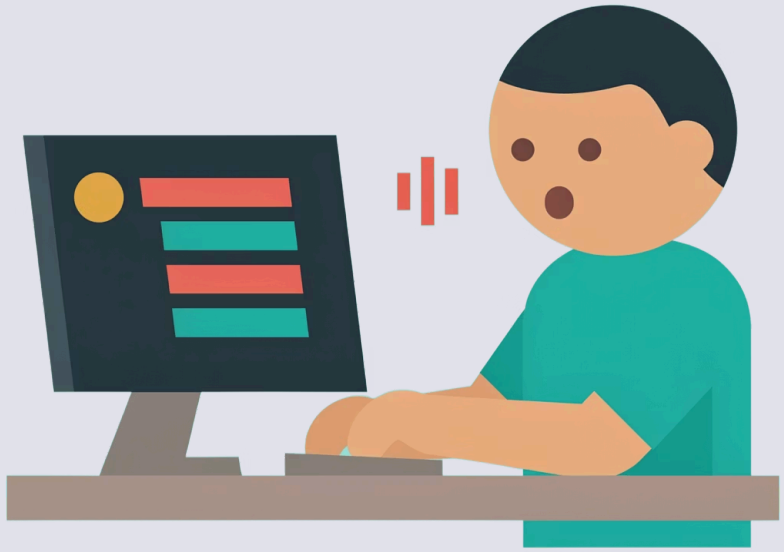
Prompting Techniques for LLMs

A practical guide for engineering students

Introduction to Generative AI

Naresh Chaurasia





No Programming Required

- LLMs such as GPT-4 and Claude do not need traditional programming
- They respond to natural language instructions called prompts
- How you structure a prompt dramatically affects the quality, accuracy, and format of the response

The Techniques We'll Cover

01

Zero-shot prompting

02

Few-shot prompting

03

Chain-of-thought prompting

04

System messages and persona setting

05

Role prompting and instruction-following patterns

06

Negative prompting

07

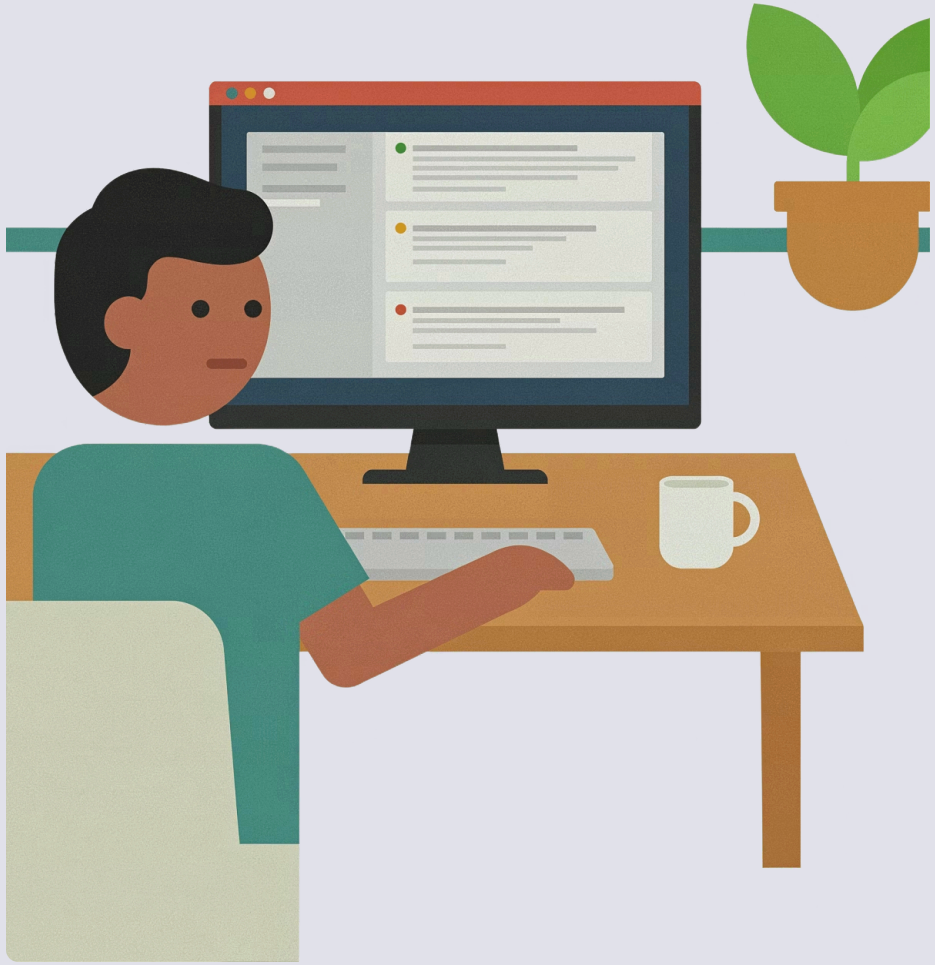
Prompt chaining

We'll watch each one change the model's behaviour on the **same problem**.

Running Case Study: Bug Report Classifier

You are building a developer tool that reads incoming **GitHub issue titles** and classifies each one.

An LLM is the classification engine. Each technique shows a more sophisticated way to prompt it.



The Three Categories

BUG

something is broken

FEATURE

a request for new functionality

QUESTION

a user asking for help or clarification

1. Zero-Shot Prompting

- A task description, and the model answers immediately
- No worked examples
- The model relies entirely on knowledge from pre-training



Zero-Shot: Example

```
Classify the following GitHub issue title as BUG, FEATURE, or QUESTION.
Issue: "App crashes when uploading a file larger than 10 MB"
Category:
```

The model is likely to answer BUG, correctly, because "crashes" is a strong signal.

Zero-Shot: When It Works

Works well when:

- The task is common and well represented in training data (sentiment, classification, translation)
- Categories and output format are self-evident from the description
- You need a **fast baseline** before investing in examples

Tends to fail on:

niche domains, ambiguous categories, or formats the model rarely sees.



2. Few-Shot Prompting

- Embed two to five worked examples (input-output pairs) in the prompt, before the real query
- The model infers the pattern from those demonstrations

Few-Shot: Example

```
Classify the following GitHub issue title as BUG, FEATURE, or QUESTION.
Examples:
Issue: "Login button unresponsive after latest deployment" -> BUG
Issue: "Add dark mode support to the dashboard" -> FEATURE
Issue: "How do I reset my API key?" -> QUESTION
Now classify:
Issue: "Export to CSV silently produces an empty file"
Category:
```

The model returns BUG.

Few-Shot: When It Works

Works well when:

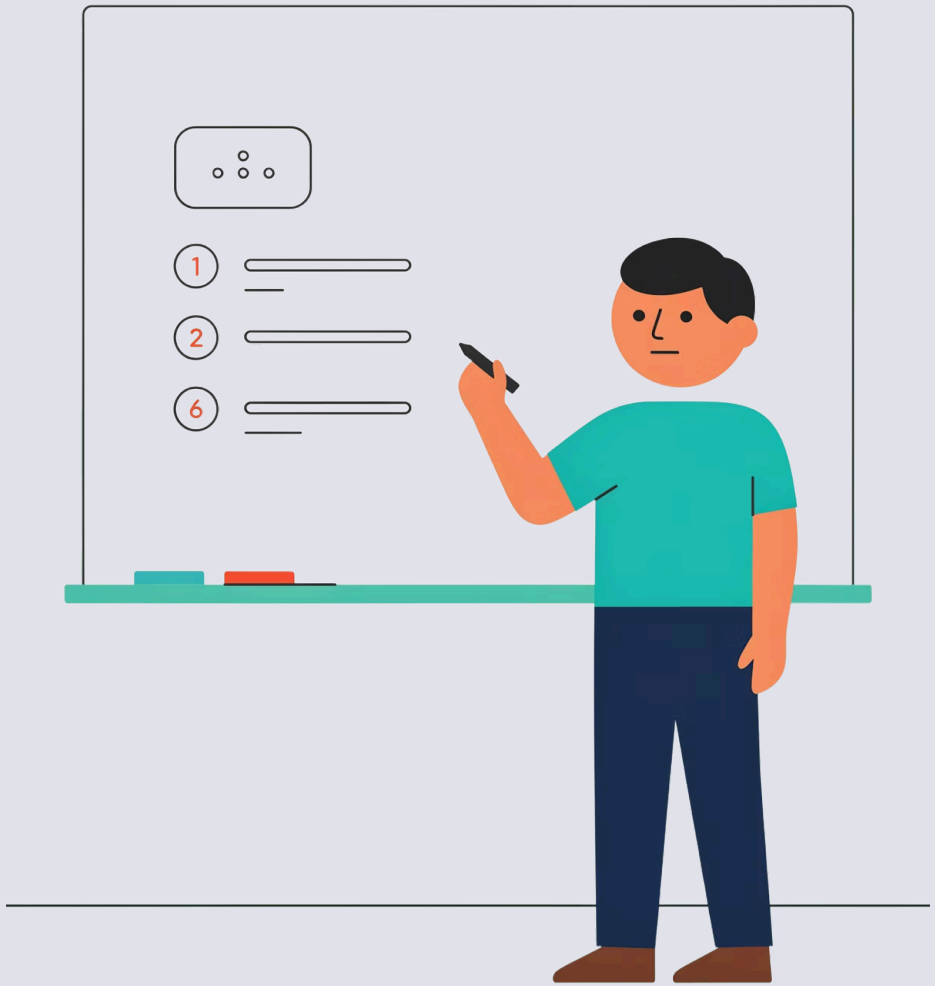
- The task has **subtle distinctions** a description cannot convey (e.g. a BUG vs a QUESTION about unexpected behaviour)
- The output must follow a **precise format** (JSON schema, code template)
- The domain is specialised and the model has weak priors

Key insight:

The examples act as an **implicit specification**. 3 to 5 usually suffice; beyond 10, the prompt gets long and returns diminish.

3. Chain-of-Thought (CoT) Prompting

- Instruct the model to reason step by step before giving a final answer
- Either say "think step by step" explicitly...
- ...or demonstrate reasoning steps in your few-shot examples



Chain-of-Thought: Example Prompt

Classify the following GitHub issue title as BUG, FEATURE, or QUESTION.
Think step by step before giving your final answer.
Issue: "Sometimes the search returns no results even though data exists"
Reasoning:

Chain-of-Thought: Example Response

Reasoning:
1. The word "sometimes" suggests intermittent behaviour, not a missing feature.
2. "Returns no results even though data exists" indicates incorrect output:
 the system is behaving contrary to expectation.
3. This is therefore a defect in existing functionality.
Category: BUG

Chain-of-Thought: When It Works

Works well when:

- The task needs **multi-step inference** (maths, logic puzzles, diagnosis)
- The answer depends on **ruling out alternatives**
- You need an **audit trail**: the steps show where the model went wrong

Less useful:

for simple pattern matching: extra tokens add latency without improving accuracy.

4. System Messages and Persona Setting

Two terms that are often confused. We'll separate them clearly:

System message

the container

Persona setting

one kind of content inside it

What Is a System Message?

- A special prompt processed before the conversation begins
- Sets the model's role, constraints, tone, and output format for the whole session
- Supported by most modern LLM APIs (OpenAI, Anthropic)
- It is the mechanism: the field in the API, or the Project Instructions box in the Claude app

What Is Persona Setting?

One type of content you can place inside the system message

It answers: "Who is the model supposed to be?"

If you extracted only the first line below, that line alone would be the persona setting. The full block is the system message.

A System Message Carries Several Instructions

```
You are a senior software engineer with 10 years of backend experience. <-- persona setting
Classify GitHub issues as BUG, FEATURE, or QUESTION. <-- task definition
Respond with only the label, no explanation. <-- output format rule
Do not classify issues outside software development. <-- behavioural constraint
```

Container and Content

```
System Message <-- the envelope
|
+-- Persona setting ("You are a senior software engineer...")
+-- Output format rules ("Always respond in JSON...")
+-- Behavioural limits ("Never discuss competitor products...")
+-- Domain context ("This tool is used for GitHub triage...")
```

System Message vs Persona Setting

	System Message	Persona Setting
What it is	A technical input field	A writing technique / content choice
What it does	Delivers persistent instructions	Assigns a role or identity
Scope	Can contain many things	Just one aspect of the system message
Can exist without the other?	Yes: a system message with no persona is valid	No: a persona must be delivered through a system message

Two Ways to Set a System Message

Via the API

(applications and pipelines)

Pass a "system" field in the API request.

Via the Claude app

(no code)

Create a **Project** in claude.ai and write your instructions in **Project Instructions**. Every conversation in that Project applies them from the first message.

API Example

```
{
 "system": "You are an expert software triage assistant. Your job is to classify
 GitHub issue titles. Always respond with a single JSON object containing
 two keys: 'category' (one of BUG, FEATURE, QUESTION) and 'confidence'
 (a number between 0 and 1). Do not include any other text.",
 "messages": [
 {
 "role": "user",
 "content": "Issue: \"Search bar freezes the browser tab on mobile\""
 }
 ]
}
```

API Example: Expected Response

```
{
  "category": "BUG",
  "confidence": 0.95
}
```

Same system message, every user turn: **no repeating instructions**.

Why Structure Matters

The system message gives **persistent context** for every turn:

Output format control

enforce JSON, XML, or a schema so code can parse results

Domain framing

software issues, medical records, legal contracts

Behavioural guardrails

no unsolicited caveats, apologies, or disclaimers

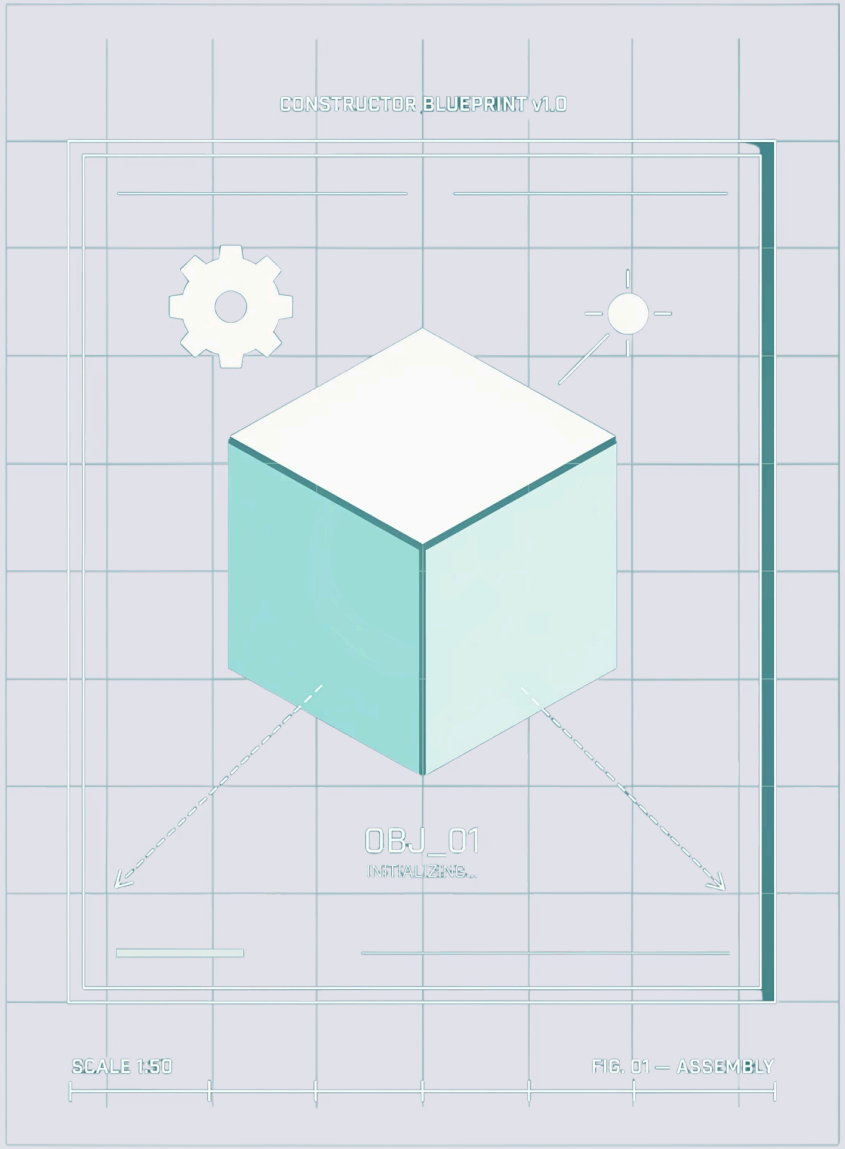
Tone and register

formal, concise, domain terminology

Analogy: A Constructor

The system message is like a **constructor** in object-oriented programming.

- It initialises the object (the model session) into a specific state
- **Before** any methods (user messages) are called



System Message vs Role Prompting

Both involve telling the model who to be. They solve **different problems**.

System message / persona

"How do I configure the model's identity and behaviour before the conversation starts?" Setup and persistence.

Role prompting

"How do I invoke domain expertise to improve one specific response?" Activating the right knowledge and reasoning style.

Side by Side

	System Message / Persona	Role Prompting
Where it lives	System message field, before the conversation	Inside a user or system message
Primary purpose	Configure the session: format, constraints, identity, scope	Activate domain-relevant reasoning and vocabulary
Is persona the goal?	No: one tool among several	Yes: the main technique
Scope	Entire conversation	Often a single task
Example concern	"My API must always return JSON and never add caveats"	"Reason like a QA engineer, not a generalist"

Analogy: Hiring a Contractor

System message and persona

= the **employment contract**: job title, working hours, what they may do, report format, department.

Role prompting

= "put your **QA hat** on for this one": a particular mode of thinking for a particular task.

In practice they are **combined**: the system message sets a base persona; role prompting fine-tunes the reasoning for a sub-task.



5. Role Prompting and Instruction Patterns

- **Role prompting: assign the model an explicit persona or professional identity**
- **Instruction-following patterns: structural conventions (numbered steps, conditional logic, output templates) that help the model parse complex directives reliably**

Role Prompting: Example

```
You are a senior software engineer with ten years of experience in backend systems.
A junior developer has submitted the following GitHub issue.
Your task is to classify it and write a one-sentence triage note explaining your decision.
Issue: "API returns 200 OK but the response body is always null"
```

- The **role** steers vocabulary toward domain-appropriate language
- The **instruction** constrains the output shape

Instruction-Following Patterns

When a task has several sub-steps, **enumerate them**:

```
You are a triage bot. For each issue title provided, do the following in order:
1. Identify the key symptom or request in the title.
2. Determine whether it describes broken behaviour, missing functionality,
 or a user seeking information.
3. Output exactly one of: BUG, FEATURE, or QUESTION.
4. Do not output anything else.
Issue: "Pagination does not work when filters are applied"
```

Why Numbered Instructions Help

- They reduce the chance the model skips a step or conflates two sub-tasks
- "Do not output anything else" is also an example of negative prompting, which is next



Why Role Prompting Works

- LLMs are trained on text written by real people in real roles
- A role like "senior engineer" or "QA analyst" biases the model toward that role's vocabulary, assumptions, and reasoning patterns
- Especially effective when you want professional judgement, not surface-level pattern matching

6. Negative Prompting

Telling the model explicitly what it must **NOT** do.

A complement to positive instructions, used to eliminate common failure modes:

Unnecessary preamble

("Sure! Happy to help...")

Caveats that break parsers

("Note: I am an AI and...")

Multiple formats when only one is required

Hallucinated categories outside the allowed set

Negative Prompting: Example

```
You are a triage classifier. Classify the GitHub issue title below.
Rules:
- Output only one of the following labels: BUG, FEATURE, QUESTION.
- Do NOT output any explanation or reasoning.
- Do NOT include punctuation after the label.
- Do NOT say "I cannot determine" or similar hedges.
Choose the best match.
Issue: "Will you add support for OAuth 2.0 login?"
```

Expected output:

```
FEATURE
```

Without the Constraints

> Sure! Based on the issue title, this appears to be a feature request. The user is asking for new OAuth 2.0 login support, which does not currently exist in the system. I would classify this as: FEATURE.

⚠ **That verbose response breaks any code that expects a bare label.**

Why Negative Prompting Works

LLMs are trained to be helpful and thorough, so they add context, caveats, and politeness. Negative prompting **overrides those defaults**.

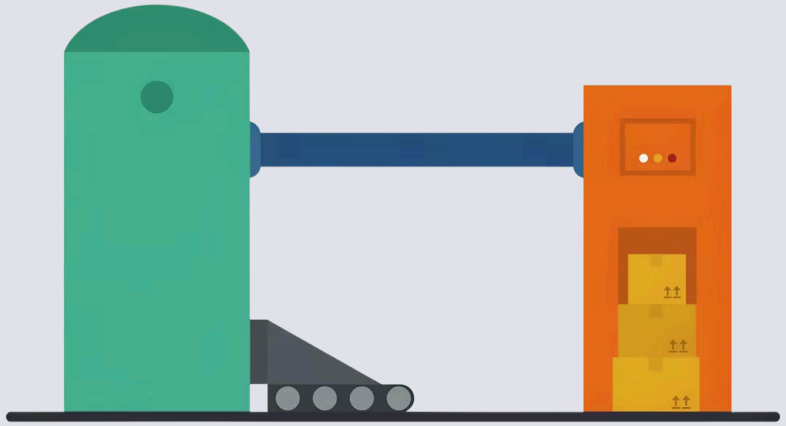
Use it when:

- Output is consumed **programmatically** and extra text causes parse errors
- Latency matters and every token counts
- The model keeps producing an unwanted pattern despite positive instructions

Pro Tip: Pair It With Few-Shot

Combine negative prompting with few-shot examples.

If every example shows **bare output**, the model internalises the pattern rather than relying on the instruction alone.



7. Prompt Chaining

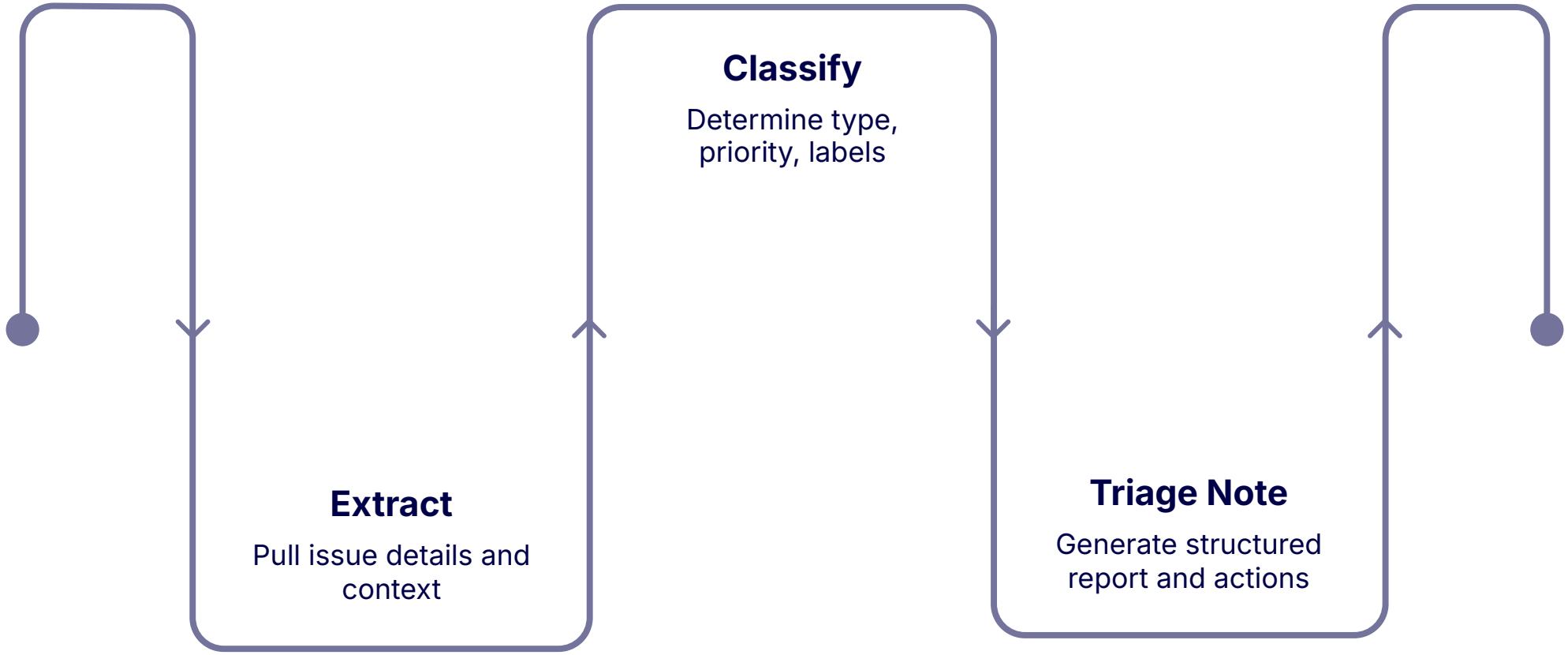
- **Break a complex task into a sequence of smaller, well-defined sub-tasks**
- **Each is handled by a separate LLM call**
- **The output of one call becomes the input of the next**

This mirrors **separation of concerns**: a pipeline of focused components, not one monolithic function.

Case Study: Automated Issue Triage Pipeline

Goal: not only classify issues, but also generate a **structured triage report**.

One prompt for everything is brittle. A chain makes each step reliable and testable.



Chain Step 1: Extract the Core Problem

```
System: You are a technical analyst. Extract the core technical problem from the
        GitHub issue title in one concise sentence. Do not classify it yet.
User: "After the v2.3 upgrade, authenticated users are randomly logged out mid-session"
```

Output (passed to Step 2):

```
Authenticated users are being unexpectedly logged out mid-session after a version upgrade.
```

Chain Step 2: Classify

```
System: You are a triage classifier. Classify the input as BUG, FEATURE, or QUESTION.
        Output only the label.
User: "Authenticated users are being unexpectedly logged out mid-session after a version upgrade."
```

Output (passed to Step 3):

```
BUG
```

Chain Step 3: Generate the Triage Summary

System: You are a software team lead writing internal triage notes. Given a category and a problem description, write a two-sentence triage note for the engineering team.
User: Category: BUG
Problem: Authenticated users are being unexpectedly logged out mid-session after a version upgrade.

Chain Step 3: Output

> This is a regression introduced in v2.3 that disrupts active user sessions, which is high-severity and should be prioritised for immediate investigation. Likely candidates include session token handling, cookie expiry configuration, or an authentication middleware change in the upgrade diff.

Why Chain Rather Than Combine?

Single monolithic prompt	Prompt chain
Hard to debug: unclear which sub-task failed	Each step is independently testable and observable
One formatting error breaks the whole output	Failures are isolated; fix only the affected step
Token budget consumed by unrelated context	Each call uses only the context relevant to its sub-task
Hard to swap or improve one component	Any step can be replaced without touching the rest
Accuracy degrades on complex multi-step tasks	Focused prompts keep the model on task

When to Use Prompt Chaining

Use it when:

- The task has clearly **separable phases** (extract, classify, generate, validate)
- Intermediate outputs must be **inspected, logged, or branched on** in code
- One prompt would exceed the model's effective attention span for complex reasoning
- Different steps benefit from **different personas or system messages**

Avoid it

for tasks one well-structured prompt handles reliably: each call adds **latency and cost**.

Summary: The Techniques (1 of 2)

Technique	Best used when	Key benefit
Zero-shot	Task is common and unambiguous	Fast, no examples needed
Few-shot	Subtle distinctions or strict output format	Examples act as an implicit specification
Chain-of-thought	Task requires multi-step reasoning	Improves accuracy; provides an audit trail
System message / persona	Consistent behaviour needed across many turns	Sets persistent context and format once

Summary: The Techniques (2 of 2)

Technique	Best used when	Key benefit
Role prompting	Domain expertise should shape language and reasoning	Activates role-specific vocabulary and judgement
Negative prompting	Output is consumed by code, or the model adds noise	Eliminates predictable failure modes
Prompt chaining	Task is complex, multi-phase, or needs inspection	Separation of concerns; each step is testable

Key Takeaways

1. Start with zero-shot; add few-shot examples when the task is subtle or the format is strict
2. Use chain-of-thought for multi-step reasoning, not simple pattern matching
3. The system message is the container; persona is one thing inside it
4. Role prompting activates expertise for a task; the system message configures the session
5. Negative prompting removes unwanted output; pair it with examples
6. Chain prompts when a task has separable phases, but only when one prompt isn't enough



Quick Check: Discuss in Pairs

1

For "Export to CSV silently produces an empty file", why might few-shot beat zero-shot?

2

Which is the container and which is the content: system message or persona? Why can't a persona exist alone?

3

Why would "Sure! Based on the issue title..." break a program that expects BUG?

4

You want to classify an issue and then draft a reply to the user. Would you use one prompt or a chain? Why?